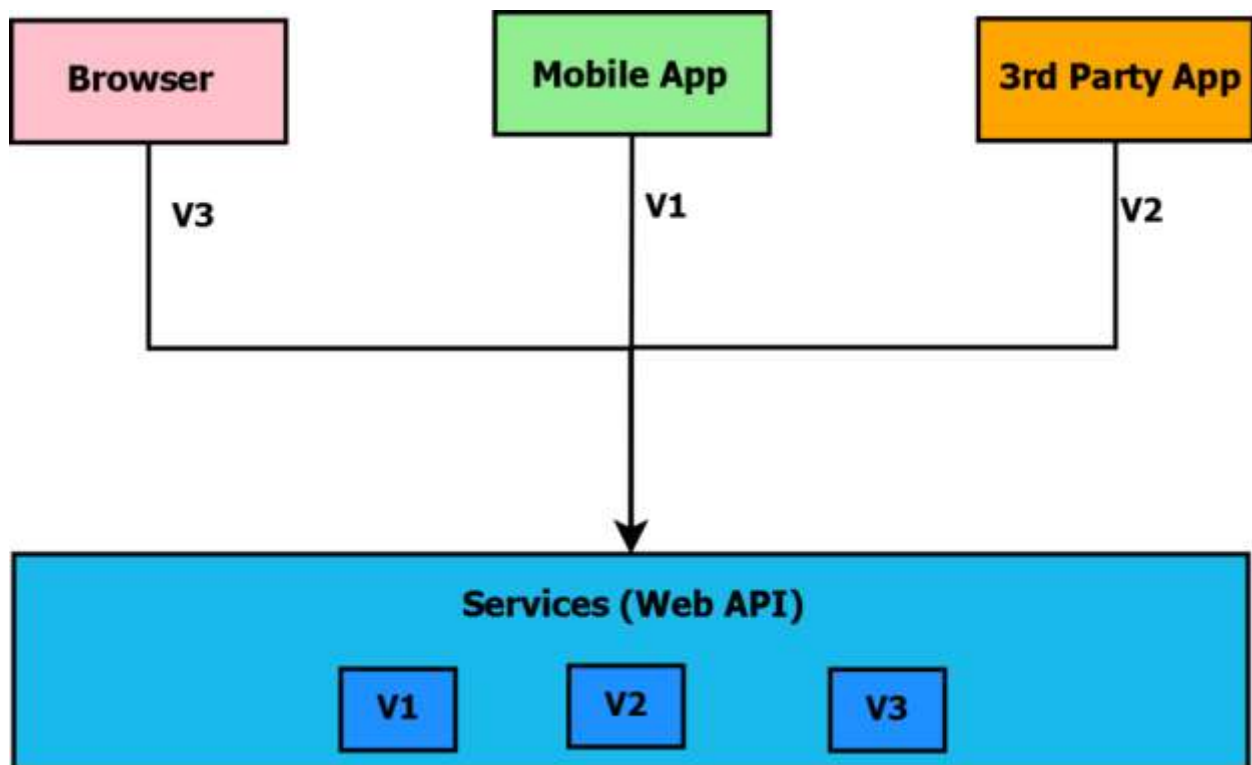
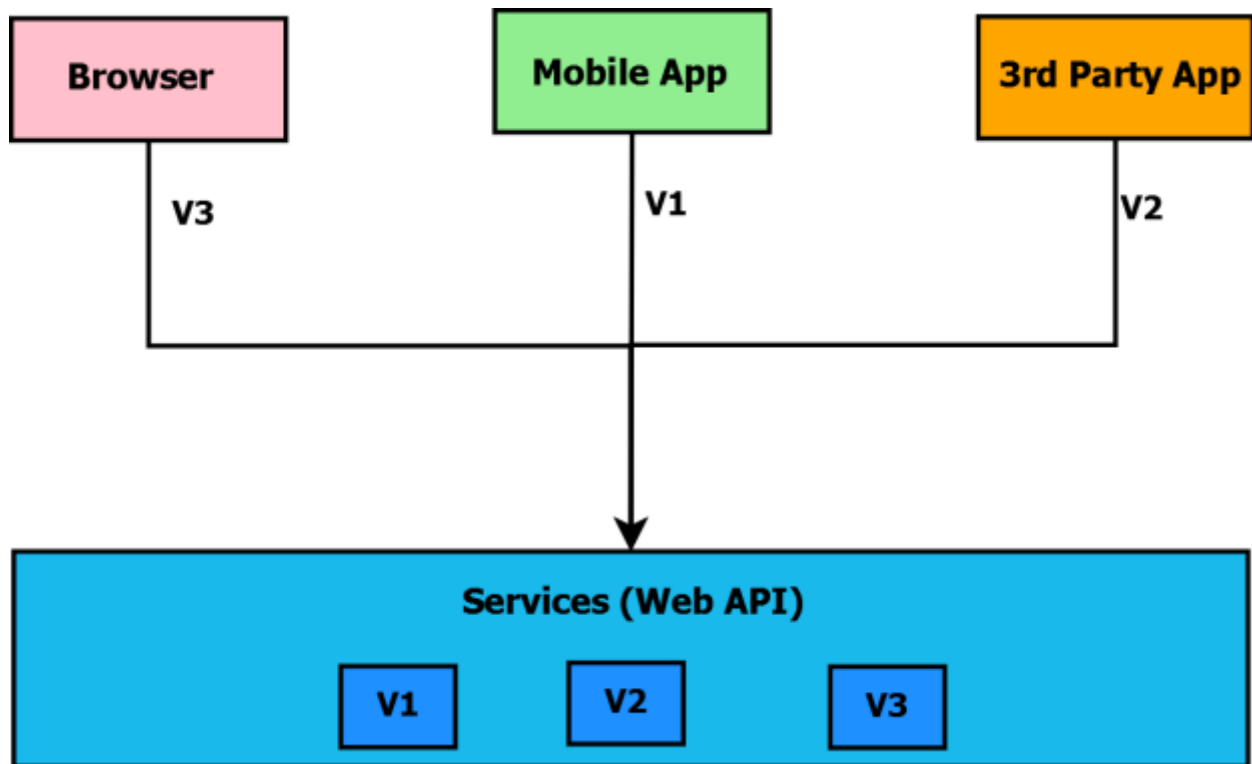


API Versioning in ASP.NET Core REST API



1. What Is API Versioning?

API Versioning is the practice of managing changes to a REST API without breaking existing client applications.

In real-world systems:

- Mobile apps
- Front-end SPAs (Angular/React)
- Third-party integrations

depend on your API. When your API evolves, **versioning ensures backward compatibility** so existing consumers continue to work.

2. Why API Versioning Is Required

Without versioning:

- A small breaking change (e.g., renaming a field) can crash production clients.
- All consumers are forced to upgrade simultaneously (impractical).

With versioning:

- Existing clients continue using **v1**
 - New clients adopt **v2**
 - Migration happens gradually
-

3. Common Scenarios That Require a New API Version

Create a new version when you:

- Remove or rename request/response properties
- Change data types (e.g., int → string)
- Modify validation rules
- Change business behavior
- Remove or replace endpoints

Do **not** create a new version for:

- Bug fixes
 - Performance improvements
 - Non-breaking optional fields
-

4. API Versioning Approaches in ASP.NET Core

ASP.NET Core supports multiple versioning strategies.

5. Versioning Methods (Most Common)

5.1 URL-Based Versioning (Most Popular)

Example

GET /api/v1/products

GET /api/v2/products

Controller

```
[ApiController]
```

```
[ApiVersion("1.0")]
```

```
[Route("api/v{version:apiVersion}/products")]
```

```
public class ProductsController : ControllerBase
```

```
{
```

```
}
```

Pros

- Easy to understand
- Visible in browser, logs, Swagger
- Most commonly used in enterprises

Cons

- URL changes when version changes

5.2 Query String Versioning

Example

GET /api/products?api-version=1.0

Configuration

```
options.ApiVersionReader = new QueryStringApiVersionReader("api-version");
```

Pros

- Clean URLs
- Easy to test

Cons

- Less explicit
 - Often disliked by REST purists
-

5.3 Header-Based Versioning

Example

GET /api/products

api-version: 2.0

Configuration

```
options.ApiVersionReader = new HeaderApiVersionReader("api-version");
```

Pros

- Clean URLs
- Strong REST compliance

Cons

- Harder to debug
 - Not browser-friendly
-

5.4 Media Type (Accept Header) Versioning

Example

Accept: application/vnd.company.v1+json

Pros

- Strict REST approach
- Highly expressive

Cons

- Complex
 - Rarely used for beginner or mid-level APIs
-

6. Recommended Approach (Instructor Guidance)

For learning, enterprise training, and real projects:

Use URL-Based Versioning

/api/v1/...

/api/v2/...

Reasons:

- Easy for students and teams
 - Swagger-friendly
 - Industry-standard for internal APIs
-

7. Setting Up API Versioning in ASP.NET Core

Step 1: Install Package

```
dotnet add package Microsoft.AspNetCore.Mvc.Versioning
```

Step 2: Configure Versioning in Program.cs

```
builder.Services.AddApiVersioning(options =>
```

```
{  
    options.DefaultApiVersion = new ApiVersion(1, 0);  
    options.AssumeDefaultVersionWhenUnspecified = true;  
    options.ReportApiVersions = true;  
});
```

This:

- Defaults to v1.0
 - Adds response headers:
 - api-supported-versions
 - api-deprecated-versions
-

Step 3: Versioned Controllers

```
[ApiController]  
[ApiVersion("1.0")]  
[ApiVersion("2.0")]  
[Route("api/v{version:apiVersion}/products")]  
public class ProductsController : ControllerBase  
{  
    [HttpGet]  
    public IActionResult GetV1() => Ok("Version 1");  
  
    [HttpGet, MapToApiVersion("2.0")]  
    public IActionResult GetV2() => Ok("Version 2");  
}
```

8. Versioning + Swagger (Important for Training)

To show multiple versions in Swagger:

- Configure Swagger per API version
- Each version appears as a dropdown

This is **critical** for:

- Student demos
 - API documentation
 - Client onboarding
-

9. Versioning Best Practices

- Start with **v1**, not v0
 - Keep versions stable
 - Deprecate old versions gradually
 - Document changes clearly
 - Avoid frequent version bumps
 - Combine with **Clean Architecture** and **Microservices**
-

10. Real-Time Example (Industry Scenario)

v1

```
{  
  "id": 1,  
  "name": "Laptop"  
}
```

v2

```
{  
  "productId": 1,  
  "productName": "Laptop",
```

"category": "Electronics"

}

v1 remains untouched

v2 introduces enhanced response
